

TradingAgents

Deployment Guide for Self-Hosted Coolify

Version 1.0 | Ubuntu VM | Coolify at **<http://192.168.0.161:8000>**

This guide walks you through deploying the TradingAgents multi-agent LLM financial trading framework on your self-hosted Coolify instance. It covers code preparation, Coolify configuration, environment variables, port mapping, and configuring API keys directly in the browser.

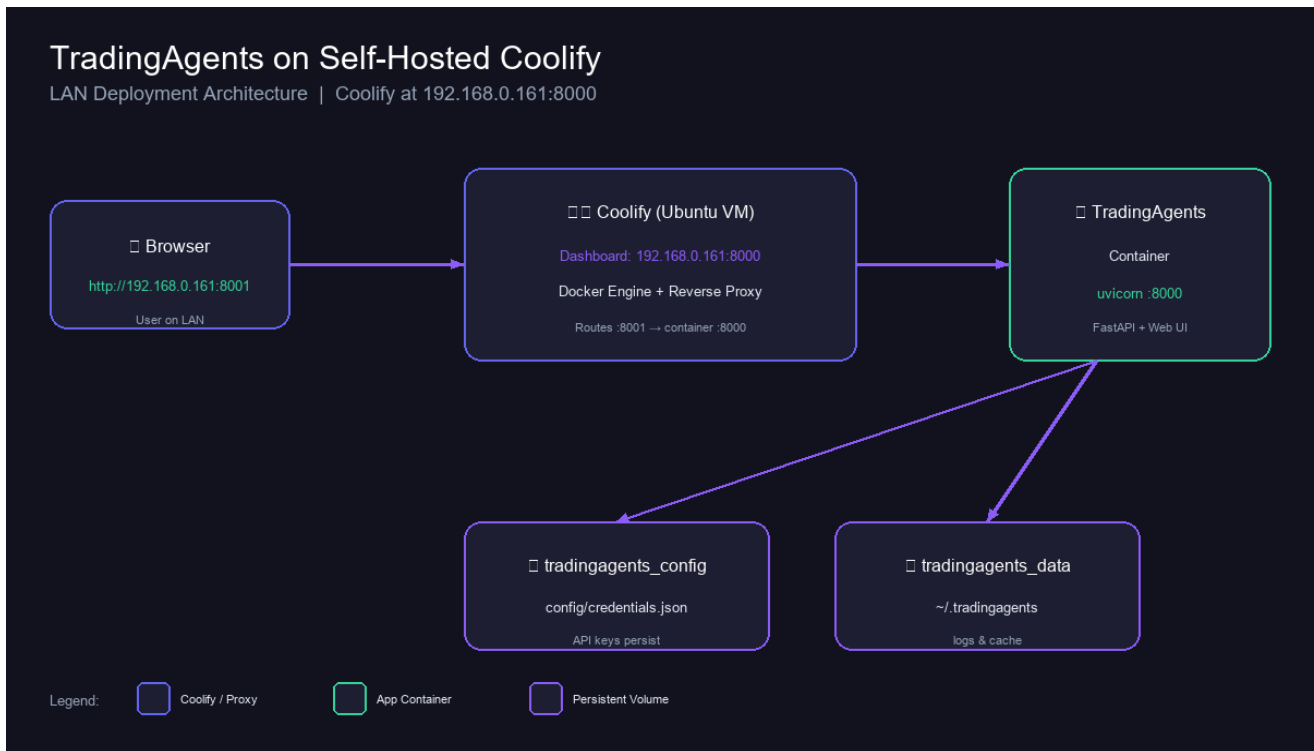
1. Prerequisites

- **Ubuntu VM** with Docker installed and running.
- **Coolify** installed and accessible at **http://192.168.0.161:8000**.
- **Git repository** (GitHub, GitLab, or Gitea) containing the TradingAgents project.
- **LLM API key** for at least one provider (OpenAI, Anthropic, Google, etc.).
- Network access from your browser to the VM on the LAN.

Note: Coolify itself occupies host port **8000**. The TradingAgents app will be exposed on host port **8001** to avoid a conflict.

2. Architecture Overview

The diagram below shows how the browser, Coolify, the app container, and persistent volumes fit together on your LAN.



Key points: The browser reaches the app at **http://192.168.0.161:8001**. Coolify's proxy routes that to the container's internal port **8000**. API keys saved in the browser persist to the **tradingagents_config** volume.

3. Required Code Changes

Two files were updated so the app runs headless in a container and avoids the port conflict with Coolify.

3.1 Dockerfile — headless web server

The original CMD launched the desktop launcher, which binds to 127.0.0.1 on a random port and tries to open a native window — this fails in a headless container. It is replaced with a uvicorn web server bound to 0.0.0.0:8000:

```
CMD ["uvicorn", "app.server:create_app", "--factory", "--host", "0.0.0.0",  
"--port", "8000"]
```

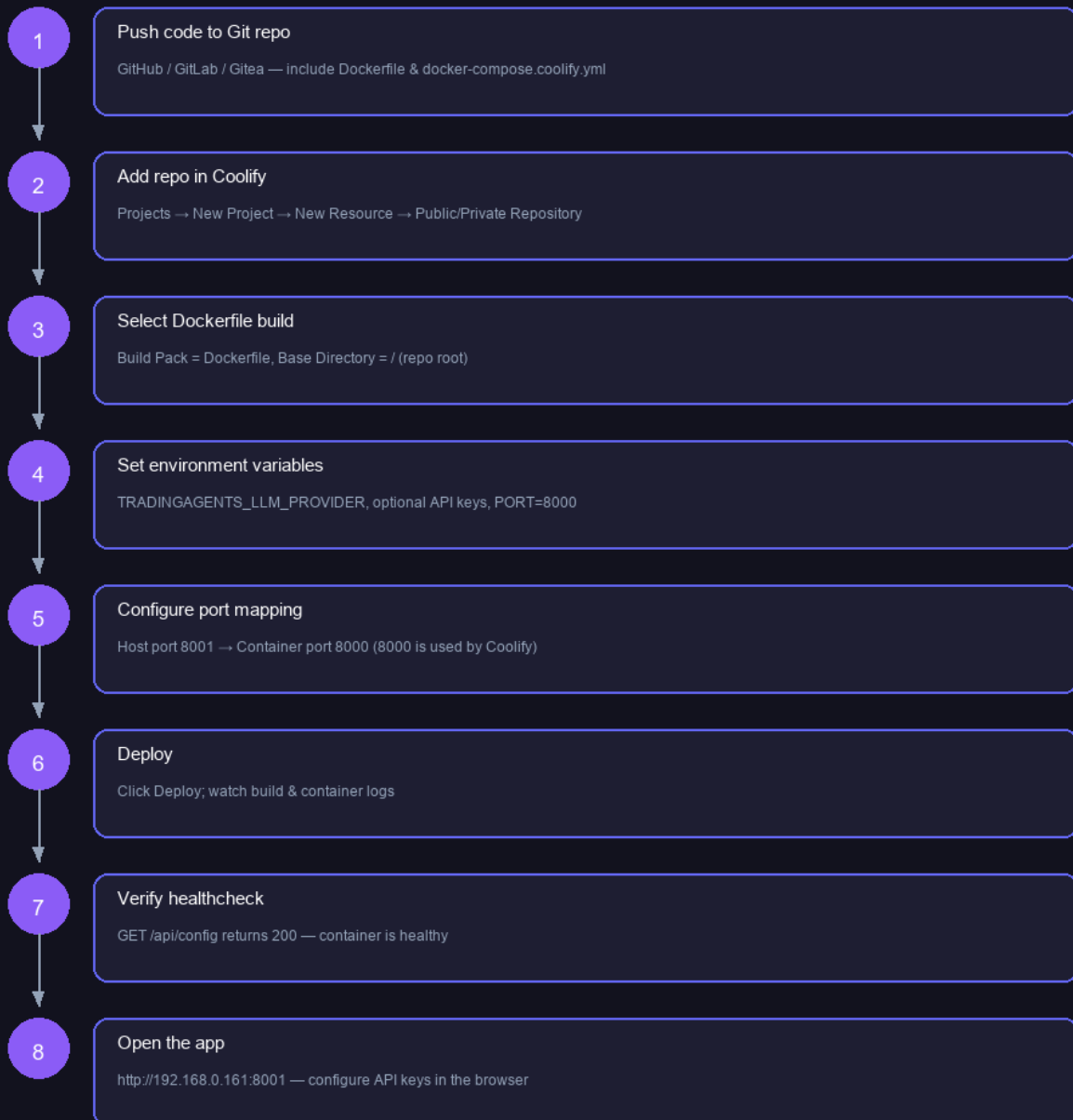
3.2 docker-compose.coolify.yml — port mapping

Since Coolify uses host port 8000, the app is mapped to host port 8001:

```
ports:  
- "8001:8000" # host 8001 -> container 8000
```

4. Step-by-Step Deployment

Deployment Flow — Step by Step



4.1 Push the code to a Git repository

- 1 Create a repository on GitHub, GitLab, or Gitea.
- 2 Push the TradingAgents project (including **Dockerfile** and **docker-compose.coolify.yml**).
- 3 Ensure **config/credentials.json** is NOT committed (it is in .gitignore).

4.2 Add the repository in Coolify

- 1 Log in to Coolify at **http://192.168.0.161:8000**.
- 2 Go to **Projects** → **New Project** → give it a name (e.g. 'TradingAgents').
- 3 Click **New Resource** → **Public Repository** (or Private Repository if using a token).
- 4 Paste the repository URL and select the branch (e.g. **main**).

4.3 Configure the build

- 1 Set **Build Pack** to **Dockerfile**.
- 2 Set **Base Directory** to **/** (repo root).
- 3 Coolify will use the **Dockerfile** at the root.

4.4 Set environment variables

In the resource's **Environment Variables** tab, add:

```
PORT=8000
TRADINGAGENTS_LLM_PROVIDER=openai
# Optional: pre-seed keys (or set them later in the browser)
# OPENAI_API_KEY=sk-...
# ANTHROPIC_API_KEY=sk-ant-...
# GOOGLE_API_KEY=AIza...
# CLOUDFLARE_ACCOUNT_ID=...
# CLOUDFLARE_GATEWAY_ID=...
# CLOUDFLARE_BYOK_ALIAS=default
```

4.5 Configure the port

In the resource's **Ports / Networking** settings, map:

```
Host port: 8001
Container port: 8000
```

This avoids the conflict with Coolify on port 8000. If you prefer, you can instead use Coolify's built-in reverse proxy with a domain.

4.6 Deploy

- 1 Click **Deploy**.
- 2 Watch the build logs — the Docker image will be built and the container started.
- 3 Wait for the status to show **Running** and the healthcheck to pass.

4.7 Verify the healthcheck

The container's healthcheck calls **GET /api/config** on port 8000. A healthy container returns HTTP 200. You can verify manually:

```
curl -f http://192.168.0.161:8001/api/config
```

4.8 Open the app

Open **http://192.168.0.161:8001** in your browser. You should see the TradingAgents configuration page.

5. Configure API Keys in the Browser

TradingAgents now lets you save API keys directly in the browser — no need to set them in Coolify. Keys persist to **config/credentials.json** inside the container, backed by the **tradingagents_config** volume.

- 1 Open **http://192.168.0.161:8001**.
- 2 In the **LLM Configuration** section, select your provider (e.g. OpenAI).
- 3 Enter your API key in the **API Key** field.
- 4 Click **Save Key** — a ✓ confirms it was saved to the project config.
- 5 Click **Validate & Fetch Models** to test the key and load available models.
- 6 Select your Deep & Quick models, then click **Launch Analysis**.

Saved keys are loaded into the environment at startup, so analysis works immediately. Keys are stored in plaintext (like .env) and are excluded from version control via .gitignore.

6. Persistent Storage

Two Docker volumes persist data across container restarts and redeploys:

Volume	Mount point	Purpose
tradingagents_config	/home/appuser/app/config	API keys & settings (credentials.json)
tradingagents_data	/home/appuser/.tradingagents	Logs, cache, memory

7. Troubleshooting

7.1 Healthcheck failing

If the container shows unhealthy, check that the app is listening on port 8000 inside the container and that the Dockerfile CMD uses uvicorn (not the desktop launcher).

7.2 Port conflict

Coolify uses port 8000. If you try to map the app to 8000 as well, the container will fail to start. Use host port 8001 (or another free port).

7.3 Cannot reach the app from browser

- Verify the VM firewall allows inbound traffic on port 8001.
- Confirm the container is running: **docker ps** on the VM.
- Check the app logs in Coolify for startup errors.

7.4 API key not working

- Ensure you clicked **Save Key** (not just typed the key).
- Verify the key is valid with **Validate & Fetch Models**.
- Check that the **tradingagents_config** volume is mounted so the key persists.

8. Security Notes

- **config/credentials.json** is excluded from git via `.gitignore` — never commit API keys.
- Keys are stored in plaintext (same as `.env`). Protect the VM and volume access.
- The app is exposed on the LAN. Restrict access with a firewall or Coolify's authentication if needed.
- Use Coolify's environment variables for non-secret config; use the browser Save Key feature for secrets.

9. Quick Reference

- Coolify dashboard: **http://192.168.0.161:8000**
- TradingAgents app: **http://192.168.0.161:8001**
- Healthcheck: **GET /api/config**
- Config store: **config/credentials.json** (volume: `tradingagents_config`)
- Build: **Dockerfile** → uvicorn on 0.0.0.0:8000